CMPS 3240 Comp Arch II: Organization
Homework 6

Noah Gallego

April 22, 2026

4.1

Instruction: `AND Rd,Rs,Rt`

Interpretation: `Reg[Rd] = Reg[Rs] AND Reg[Rt]`

4.1.1 [5] <§4.1> What are the values of control signals generated by the control in Figure 4.2 for the above instruction?

Signal	Value
RegDst	1
ALUSrc	0
MemtoReg	0
RegWrite	1
MemRead	0
MemWrite	0
Branch	0
ALUOp	10 (R-type, ALU performs AND)

4.1.2 [5] <§4.1> Which resources (blocks) perform a useful function for this instruction?

Instruction memory, Registers (read Rs, Rt and write Rd), ALU (performs AND), the adder that computes PC+4, the ALUSrc mux, the MemtoReg mux, the PCSrc mux, and the Control unit.

4.1.3 [10] <§4.1> Which resources (blocks) produce outputs, but their outputs are not used for this instruction? Which resources produce no outputs for this instruction?

Produce outputs that are not used: the branch-target adder (computes PC + offset, discarded because Branch = 0) and the ALU Zero output.

Produce no outputs: Data memory (MemRead = 0 and MemWrite = 0, so no read data is produced and no write occurs).

4.3

Datapath from Figure 4.2. Latencies: I-Mem 400 ps, Add 100 ps, Mux 30 ps, ALU 120 ps, Regs 200 ps, D-Mem 350 ps, Control 100 ps. Costs: 1000, 30, 10, 100, 200, 2000, 500.

Adding a multiplier to the ALU: +300 ps latency, +600 cost, 5% fewer instructions executed.

4.3.1 [10] <§4.1> What is the clock cycle time with and without this improvement?

The critical path is the load instruction: I-Mem → Regs → Mux → ALU → D-Mem → Mux.

Without improvement: $400 + 200 + 30 + 120 + 350 + 30 = 1130$ ps.

With improvement (ALU = 420 ps): $400 + 200 + 30 + 420 + 350 + 30 = 1430$ ps.

4.3.2 [10] <§4.1> What is the speedup achieved by adding this improvement?

$$\text{Speedup} = \frac{\text{OldTime}}{\text{NewTime}} = \frac{1130 \times N}{1430 \times 0.95N} = \frac{1130}{1358.5} \approx 0.832$$

This is a slowdown of about 17% (speedup < 1), so the change hurts performance.

4.3.3 [10] <§4.1> Compare the cost/performance ratio with and without this improvement.

Sum of component costs without improvement: $1000 + 30 + 10 + 100 + 200 + 2000 + 500 = 3840$.

With improvement (ALU cost 100 → 700): $3840 + 600 = 4440$.

Using cost/performance = cost × execution time per instruction (relative, with N instructions):

Without: $3840 \times 1130 \times N = 4{,}339{,}200\,N$.

With: $4440 \times 1430 \times 0.95N = 6{,}031{,}740\,N$.

Ratio: 6,031,740/4,339,200 ≈ 1.39. Cost/performance is about 39% worse with the improvement, so it is not worth adding.

4.4

Problems in this exercise assume that logic blocks needed to implement a processor's datapath have the following latencies:

I-Mem	Add	Mux	ALU	Regs	D-Mem	Sign-Extend	Shift-Left-2
200ps	70ps	20ps	90ps	90ps	250ps	15ps	10ps

4.4.1 [10] <§4.3> If the only thing we need to do in a processor is fetch consecutive instructions (Figure 4.6), what would the cycle time be?

Only I-Mem and the PC+4 adder are needed, running in parallel off PC. Cycle time = max(I-Mem, Add) = $\max(200, 70) = 200$ ps.

4.4.2 [10] <§4.3> For a datapath similar to the one in Figure 4.11, but for a processor that only has one type of instruction: unconditional PC-relative branch. What would the cycle time be for this datapath?

Path: I-Mem → Sign-Extend → Shift-Left-2 → Add.

Cycle time = $200 + 15 + 10 + 70 = 295$ ps.

4.4.3 [10] <§4.3> Repeat 4.4.2, but this time we need to support only conditional PC-relative branches.

Two parallel paths; cycle time is the longer one.

Branch-target path: $200 + 15 + 10 + 70 + 20 = 315$ ps.

Condition path: I-Mem → Regs → ALU → Mux = $200 + 90 + 90 + 20 = 400$ ps.

Cycle time = $\max(315, 400) = 400$ ps.

4.4.4 [10] <§4.3> Which kinds of instructions require this resource?

Branch instructions (e.g., `beq`, `bne`) and jump instructions (e.g., `j`, `jal`) require Shift-Left-2 to form the branch/jump target address.

4.4.5 [20] <§4.3> For which kinds of instructions (if any) is this resource on the critical path?

For conditional branches (`beq`), the condition path (I-Mem + Regs + ALU + Mux = 400 ps) is longer than the branch-target path (315 ps), so Shift-Left-2 is *not* on the critical path.

For unconditional jumps, the only timing path goes through Shift-Left-2, so it *is* on that instruction's critical path (though short: 210 ps).

4.4.6 [10] <§4.3> Assuming that we only support `beq` and `add` instructions, discuss how changes in the given latency of this resource affect the cycle time of the processor. Assume that the latencies of other resources do not change.

Shift-Left-2 is only used by `beq`, on its branch-target path. That path has length $200 + 15 + 70 + 20 + L = 305 + L$, where L is the Shift-Left-2 latency.

Other critical paths:

- `add`: I-Mem + Regs + Mux + ALU + Mux = $200 + 90 + 20 + 90 + 20 = 420$ ps.
- `beq` condition path: $200 + 90 + 90 + 20 = 400$ ps.

The cycle time is $\max(420, 400, 305 + L) = 420$ ps as long as $L \leq 115$ ps. Below that threshold, changes in Shift-Left-2 latency have *no effect* on cycle time. For $L > 115$ ps, the branch-target path becomes critical and cycle time grows as $305 + L$.